Conference Management System

Technology: Java, JavaFX, Maven, JDBC, SQLite, and JUnit

1. Project Overview

In this project, students, working in groups of two or three, will design and implement a desktop-based **Conference Management System** using JavaFX and Maven.

The system will allow administrator to manage conferences, participants to register for conferences, and both roles, to access the functionality permitted to them. The application must maintain accurate and consistent data in a relational database and provide a user-friendly, internationalized graphical interface.
Project aims to integrate DB programming, JDBC, object-oriented design, design patterns (2), MVC, streams & lambda features (3), internationalization and localization, data structures (4), functional programming, and S/W testing.

2. User Roles and Authentication

The application shall support two roles: **Administrator** and **Participant**; who will login to the system through entering their email and password.

2.1 Administrator

The administrator will use predefined credentials supplied by the developer. The administrator's email address, password, name, and age will be initialized as predefined profile information. Authentication credentials must be handled separately from profile information.
After successful authentication, the administrator will access an administrative control panel.

2.2 Participant

A new participant must create an account by providing the following information:

- Full name
- Email address
- Age
- Password

The system shall validate the supplied information and ensure that each email address is unique.
After creating an account, the participant can login using his email address and password. Existing participants must not be required to create a new account each time they use the application.
The application shall provide appropriate validation messages for invalid credentials and incomplete or invalid registration information.
Passwords must not be stored in plain text.

3. Conference Management

Each conference shall contain the following information:

- Conference ID: unique identifier
- Title or description
- Capacity: maximum number of confirmed participants
- Start date and time
- End date and time
- Venue or location (optional)

The system shall validate that the conference capacity is positive and that the end date and time are later than the start date and time.

3.1 Administrator Operations

The administrator shall be able to CRUD:

1. Create a conference.
2. View the complete list of conferences and their details.
3. Update conference information.
4. Delete a conference, provided that it has no confirmed participants and no participants on its waiting list.
5. View administrative reports by selecting one report at a time from a list.

The system shall prevent invalid operations, including reducing conference capacity below the number of confirmed participants.

Conference information and administrative changes must be persisted in the database.

4. Participant Operations

After logging in, a participant shall be able to:

1. View all existing conferences and their complete details.
2. View conference capacity, the number of confirmed participants, the number of available seats, and whether the conference is full.
3. Register for a conference.
4. Join a conference's waiting list when its capacity has been reached.
5. View all conferences for which he has a confirmed registration.
6. View all conferences for which he is on a waiting list.
7. Cancel a confirmed registration.
8. Cancel a waiting-list entry.
9. View and update his own profile information, subject to validation rules.

A participant shall not be permitted to register for the same conference more than once, whether as a confirmed participant or a waiting-list candidate.

5. Registration Rules and Waiting-List Management

The system shall enforce the following business rules:

5.1 Conference capacity

The number of confirmed participants must never exceed the conference's capacity.
If a conference has available seats, an eligible participant shall be registered immediately. Otherwise, the participant may join the waiting list.

5.2 Scheduling conflicts

A participant may have confirmed registrations only for conferences that do not overlap in date and time.
The same scheduling-conflict rule shall apply when a participant attempts to join a waiting list. A participant shall not be allowed to join a waiting list for a conference that conflicts with an existing confirmed registration.

Two conferences overlap when their time intervals intersect. A conference ending at the exact time another begins shall not be considered overlapping.

5.3 Waiting-list order

Each conference shall maintain its own waiting list (FIFO).
When a confirmed participant cancels their registration, the system shall automatically attempt to promote the earliest eligible participant from that conference's waiting list.
If a waiting-list participant is no longer eligible because of a scheduling conflict or another applicable constraint, the system shall skip that participant and evaluate the next candidate. The skipped participant shall remain on the waiting list unless he cancels or becomes ineligible under a separately documented rule.
When a participant is promoted, the system shall update his status and the conference's available capacity accordingly.

5.4 Cancellation

When a participant cancels a confirmed registration, the system shall update the registration and process the waiting list automatically.
When a participant cancels a waiting-list entry, the system shall remove that entry without affecting the confirmed registrations.
All related database changes must preserve data consistency, including when multiple operations affect the same conference.

6. Administrative Reports

The administrator shall be able to select and display one report at a time from a report menu.

The system shall provide at least the following reports:

1. **Conference Overview:** Conference details, capacity, confirmed registrations, and available seats.
2. **Conference Participant Report:** Confirmed participants for a selected conference.
3. **Waiting-List Report:** Participants waiting for a selected conference, in queue order.
4. **Participant Registration Report:** Confirmed and wait-listed conferences for a selected participant.
5. **Conference Occupancy Report:** The number and percentage of seats occupied in each conference.
6. **Unregistered Conferences Report:** Conferences with no confirmed participants and no waiting-list entries.

Reports must be generated from database records and display accurate, up-to-date information. The interface shall allow the administrator to return to the report selection menu.

7. Technical and Architectural Requirements

7.1 JavaFX and Maven

The application must be implemented in Java using JavaFX for the graphical user interface and Maven for project and dependency management.
The project must have a clear and maintainable structure, with appropriate separation of user-interface code, business logic, and database operations.

7.2 JDBC and Relational Database

Students shall create and populate a local SQLite database and connect the application to it using JDBC.
The database shall persist user profiles, conferences, registrations, and waiting-list information.
Students must implement the required CRUD operations and use appropriate primary keys, foreign keys, unique constraints, transactions, and other database mechanisms to maintain data integrity.

7.3 MVC Architecture

Students shall apply the Model-View-Controller (MVC) architectural pattern.
The Model shall represent application data and business rules. The View shall provide the JavaFX interface, and the Controller shall handle user interactions and coordinate application services.
Business rules must not be implemented in JavaFX controllers (which are for controlling the UIs). Database access and business logic should be organized into appropriate, separate classes.

7.4 Design Patterns

Students must implement **at least two design patterns from distinct design categories** and document their purpose and implementation. For each pattern, students shall identify its category, explain the problem it addresses, provide evidence of its implementation, and justify why it is appropriate for the application.
The MVC architecture alone shall not satisfy this requirement.

7.5 Internationalization and Localization

The application must support at least two locales, such as English (Canada) and French (Canada). Students shall use appropriate Java internationalization classes.
All user-facing interface text, menus, buttons, validation messages, error messages, and report headings must be available in both supported languages.
Users must be able to switch the application language without changing the underlying data. Date, time, and number formatting shall follow the selected locale where appropriate.

7.6 Data Structures, Streams, and Lambda Expressions

Students shall select suitable data structures for conferences, users, registrations, and waiting lists.
They must demonstrate meaningful use of the Java Streams, lambda expressions, or method references in operations such as filtering, sorting, grouping, or generating reports.
The chosen data structures and processing techniques must be justified in the project documentation.

7.7 User Interface

The application shall provide a login interface, and separate interfaces for administrators and participants.
The interface must clearly display relevant information, provide navigation between screens, and present understandable validation and error messages.
The application shall prevent unauthorized access to administrative functionality.

8. Testing and Quality Assurance

Students shall use JUnit to develop automated tests for the application. Tests must cover, at a minimum:

- Valid and invalid authentication.
- Participant account creation and duplicate email addresses.
- Conference creation, modification, and deletion.
- Invalid conference capacity and date-time values.
- Registration when seats are available.
- Registration when a conference is full.
- Duplicate registrations.
- Scheduling conflicts between conferences.
- Cancellation of confirmed registrations.
- Automatic waiting-list promotion and FIFO ordering.
- Waiting-list promotion when the earliest candidate is ineligible.

- Role-based access restrictions.
- Locale switching and locale-sensitive formatting.

Students shall document their testing approach and provide evidence of successful test execution. Appropriate unit tests and database integration tests must be included.